

MACHINE LEARNING PROJECT EXPLANATION

Insurance Cost Prediction Using Random Forest

A practical guide to the dataset, Random Forest Regression workflow, hyperparameter search, model benefits, evaluation, and reliable fixes.

Based on the supplied Random Forest notebook export
Dataset: **insurance.csv**

1. Project Summary

This project predicts medical insurance expenses from demographic and lifestyle information. It uses **Random Forest Regression**, an ensemble method that trains many decision trees and averages their predictions.

Why an ensemble? A single decision tree can be unstable and overfit. A Random Forest combines many trees trained with randomized samples and feature choices, usually producing a more stable prediction.

Input features

- Age
- BMI
- Number of children
- Sex
- Smoking status
- Region

Prediction target

Expenses

A continuous insurance-cost value predicted for each person.

End-to-end workflow

1. Load `insurance.csv` with pandas.
2. Convert categorical columns into one-hot encoded features.
3. Separate input features from the `expenses` target.
4. Split the data into training and test sets.
5. Define a Random Forest Regressor and its hyperparameter ranges.
6. Train one forest for every parameter combination.
7. Calculate an R^2 score for every model.
8. Sort the results and save them to an Excel file.
9. Select a strong configuration for final predictions.

2. Code Explanation

Loading and encoding the data

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

dataset = pd.read_csv("insurance.csv")
dataset = pd.get_dummies(dataset, drop_first=True)

independent = dataset[
    ["age", "bmi", "children", "sex_male", "smoker_yes",
     "region_northwest", "region_southeast", "region_southwest"]
]
dependent = dataset["expenses"]
```

The categorical columns are converted to numeric indicator columns. Dropping the first dummy avoids redundant categories. The target should be a one-dimensional Series, which prevents the repeated `DataConversionWarning` visible in the source export.

Splitting the data

```
from sklearn.model_selection import train_test_split

x_train, x_test, y_train, y_test = train_test_split(
    independent,
    dependent,
    test_size=0.2,
    random_state=42
)
```

The forest learns from the training subset and is measured on unseen test data. Keeping the split fixed makes parameter comparisons reproducible.

Creating one Random Forest

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score

regressor = RandomForestRegressor(
    n_estimators=200,
    criterion="squared_error",
    max_depth=None,
    min_samples_split=2,
    min_samples_leaf=1,
    max_features=1.0,
    bootstrap=True,
    random_state=42,
    n_jobs=-1
)

regressor.fit(x_train, y_train)
y_pred = regressor.predict(x_test)

score = r2_score(y_test, y_pred)
print(f"R² score: {score * 100:.2f}%")
```

`n_estimators` controls the number of trees. More trees can stabilize the result but increase training time. `n_jobs=-1` allows parallel CPU work.

3. Hyperparameter Search

The notebook manually tests combinations of the following values and stores each result in a list before converting it into a DataFrame:

Parameter	Values tested	What it controls
<code>n_estimators</code>	50, 100, 200	Number of decision trees in the forest.
<code>criterion</code>	squared_error, absolute_error, friedman_mse, poisson	How the quality of a split is measured.
<code>max_depth</code>	None, 5, 10, 15, 20	Maximum depth of each tree.
<code>min_samples_split</code>	2, 5, 10	Minimum samples required to split a node.
<code>min_samples_leaf</code>	1, 2, 4	Minimum samples required in a final leaf.
<code>max_features</code>	1.0, sqrt, log2	Features considered when each split is selected.
<code>bootstrap</code>	True, False	Whether trees train on bootstrap samples.

How the experiment works

```
from itertools import product

results = []

for values in product(
    n_estimators_values,
    criterion_values,
    max_depth_values,
    min_samples_split_values,
    min_samples_leaf_values,
    max_features_values,
    bootstrap_values
):
    n_estimators, criterion, max_depth, min_samples_split, \
        min_samples_leaf, max_features, bootstrap = values

    model = RandomForestRegressor(
        n_estimators=n_estimators,
        criterion=criterion,
        max_depth=max_depth,
        min_samples_split=min_samples_split,
        min_samples_leaf=min_samples_leaf,
        max_features=max_features,
        bootstrap=bootstrap,
        random_state=42,
        n_jobs=-1
    )

    model.fit(x_train, y_train)
    y_pred = model.predict(x_test)

    results.append({
```

```
"n_estimators": n_estimators,  
"criterion": criterion,  
"max_depth": max_depth,  
"min_samples_split": min_samples_split,  
"min_samples_leaf": min_samples_leaf,  
"max_features": max_features,  
"bootstrap": bootstrap,  
"R2": r2_score(y_test, y_pred)  
})
```

The parameter grid contains $3 \times 4 \times 5 \times 3 \times 3 \times 3 \times 2 = \mathbf{3,240 \text{ combinations}}$. That is a large experiment, so parallel processing and cross-validation strategy should be planned carefully.

4. Random Forest Benefits

Benefit	Why it matters for insurance expenses
Reduces single-tree overfitting	Averaging many trees generally reduces variance and gives more stable predictions than one decision tree.
Captures nonlinear effects	Insurance expenses can change sharply with smoking status, age, BMI, and interactions between them.
Handles feature interactions	The forest can learn combinations such as smoker status interacting with age or BMI without manual interaction terms.
No standardization required	Tree-based splits use thresholds, so different feature scales do not require StandardScaler.
Supports feature importance	Feature importance can provide an initial view of which inputs contribute most to the predictions.
Parallel training	Independent trees can be trained across CPU cores with <code>n_jobs=-1</code> .

Limitations and trade-offs

- Large forests use more memory and take longer to train.
- They are harder to explain than a single decision tree or linear model.
- Feature importance can be biased; permutation importance is a useful complement.
- Random Forests do not extrapolate smoothly outside the target range observed during training.
- A single holdout R^2 is not enough to select the final model.
- The `poisson` criterion requires a non-negative target, which is valid for ordinary insurance expenses but should still be checked.

Practical recommendation: use cross-validation or `RandomizedSearchCV` for a faster, more reliable search, then evaluate the selected model with R^2 , MAE, and RMSE.

5. Corrected and Reusable Implementation

This version uses a one-dimensional target, a reproducible split, and a focused search. It also saves the final model for future predictions.

```
import joblib
import pandas as pd

from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.model_selection import RandomizedSearchCV, train_test_split

dataset = pd.read_csv("insurance.csv")
dataset = pd.get_dummies(dataset, drop_first=True)

X = dataset[
    ["age", "bmi", "children", "sex_male", "smoker_yes",
     "region_northwest", "region_southeast", "region_southwest"]
]
y = dataset["expenses"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

parameter_grid = {
    "n_estimators": [100, 200, 300],
    "criterion": ["squared_error", "absolute_error"],
    "max_depth": [None, 10, 20],
    "min_samples_split": [2, 5, 10],
    "min_samples_leaf": [1, 2, 4],
    "max_features": [1.0, "sqrt", "log2"],
    "bootstrap": [True, False]
}

search = RandomizedSearchCV(
    RandomForestRegressor(random_state=42, n_jobs=-1),
    parameter_distributions=parameter_grid,
    n_iter=30,
    scoring="r2",
    cv=5,
    random_state=42,
    n_jobs=-1
)

search.fit(X_train, y_train)
model = search.best_estimator_
y_pred = model.predict(X_test)

print("Best parameters:", search.best_params_)
print("R² score:", r2_score(y_test, y_pred))
print("MAE:", mean_absolute_error(y_test, y_pred))
print("RMSE:", mean_squared_error(y_test, y_pred) ** 0.5)

joblib.dump(model, "insurance_random_forest.joblib")
```

Randomized search samples a manageable number of configurations instead of evaluating all 3,240 combinations. Five-fold cross-validation also makes model selection less dependent on one test split.

6. Fixes Before Publishing

Issue	Correction
Column-vector warning	Replace <code>dataset[["expenses"]]</code> with <code>dataset["expenses"]</code> , or use <code>y_train.ravel()</code> before fitting.
Very large manual grid	Use <code>RandomizedSearchCV</code> or a smaller, staged grid before expanding the search.
Holdout-only selection	Use cross-validation for tuning, keeping the final test set untouched until the end.
Parallel warnings	The <code>sklearn.utils.parallel.delayed</code> messages are warnings from the parallel execution path, not prediction failures. Updating scikit-learn and using its documented parallel APIs can reduce them.
Reproducibility	Keep <code>random_state=42</code> , record library versions, and save the selected parameters.
Deployment	Preserve the exact encoded feature columns and order when making predictions on new data.

Conclusion

Random Forest Regression is a strong choice for insurance-cost prediction because it learns nonlinear relationships and interactions while reducing the instability of a single decision tree. The supplied notebook demonstrates a broad parameter search; the next improvement is to make that search reproducible, cross-validated, and computationally focused.

Prepared from the supplied Random Forest PDF export. The final displayed run reports an R^2 of approximately 80.52%; this should be rechecked after applying the corrected target shape and validation workflow.